DOCUMENTACIÓN TÉCNICA Y FUNCIONAL

Contexto del Proyecto: TecnoStock Web

Sistema Web Fullstack de Gestión de Inventario, Entradas de Proveedores, Ventas y Kardex en Tiempo Real

> **Nombre del Sistema:** TecnoStock Web — Gestión de Inventario para Tiendas de Tecnología
> **Repositorio Oficial en GitHub:** https://github.com/PulidoMiguelacademy/Gestion_de_inventario.
> **Stack Tecnológico:** React 19 + Vite 8 (Frontend) | Python 3 + FastAPI (Backend Analítico) | Google Firebase Auth & Cloud Firestore (Base de Datos en Tiempo Real)

1. Introducción y Problemática del Negocio

En los pequeños y medianos comercios dedicados a la comercialización de productos tecnológicos (teléfonos celulares, computadores, repuestos y accesorios), el control de existencias suele llevarse en hojas de cálculo manuales o cuadernos físicos. Esta práctica genera cuatro problemas críticos:

1. **Desabastecimiento y pérdida de ventas:** No existen alertas automáticas cuando un artículo de alta rotación está próximo a agotarse.
2. **Descuadres de inventario:** Las ventas y las entradas de mercancía no se descuentan ni suman en tiempo real, provocando diferencias entre el stock físico en vitrina y el registrado.
3. **Falta de trazabilidad por serial / IMEI:** En dispositivos como celulares, resulta indispensable saber exactamente qué número de serie ingresó con qué proveedor y a qué cliente fue vendido para responder por garantías.
4. **Riesgo de seguridad operativa:** Todos los empleados suelen tener acceso a modificar precios o borrar registros sin una separación de permisos entre el propietario (*Administrador*) y los vendedores (*Operadores*).

TecnoStock Web nace como una plataforma centralizada, intuitiva y en tiempo real que digitaliza de principio a fin el flujo de inventario, recepción de proveedores, facturación con lector de código de barras y auditoría financiera.

2. Objetivos del Proyecto

Objetivo General

Desarrollar un sistema web Fullstack para la gestión de inventario y control de ventas en tiempo real para una tienda de tecnología, garantizando la trazabilidad de existencias, el control de acceso basado en roles y la generación de reportes financieros.

Objetivos Específicos

- **Controlar el acceso y la seguridad:** Implementar autenticación cifrada, recuperación de contraseña por correo y separación de roles (*Administrador* vs. *Operador*).
- **Centralizar el catálogo tecnológico:** Registrar, buscar, actualizar y clasificar productos con soporte de códigos de barras, precios en formato local (COP) y manejo de seriales/IMEIs.
- **Auditar las entradas y salidas (Kardex):** Registrar ingresos de mercancía asociados a proveedores y descontar automáticamente las unidades vendidas, almacenando el historial inmutable de cada movimiento.
- **Prevenir quiebres de stock:** Automatizar un semáforo de existencias (*OK*, *Bajo Stock* y *Agotado*) basado en umbrales mínimos configurables por producto.
- **Soporter la toma de decisiones financieras:** Integrar un microservicio en Python para calcular la valoración contable del inventario, el margen de ganancia proyectado y el cierre diario de caja.

3. Arquitectura Tecnológica (Stack Fullstack)

El proyecto implementa una arquitectura moderna desacoplada en tres capas principales conectadas en tiempo real:

💻 1. Frontend Web (Puerto 5173)

- **React 19 + Vite 8:** Interfaz SPA de alta velocidad.
- **AuthContext & RBAC:** Control de sesión y permisos según rol.
- **ErrorBoundary:** Protección contra caídas de renderizado.

☁️ 2. Google Firebase (Nube BaaS)

- **Firebase Authentication:** Gestión de credenciales y recuperación por correo.
- **Cloud Firestore NoSQL:** Sincronización instantánea vía `onSnapshot`.

🐍 3. Backend Analítico (Puerto 8000)

- **Python 3 + FastAPI:** Microservicio REST de alto rendimiento.
- **Motor Financiero:** Valoración de stock, márgenes y cierre diario.
- **Swagger UI:** Documentación interactiva en `/docs`.

Capa	Tecnología	Propósito en el Proyecto
Frontend	React 19, Vite 8, CSS Moderno, Lucide Icons	Interfaz de usuario reactiva, rápida y adaptada al flujo de mostrador y bodega.
Autenticación	Firebase Authentication	Inicio de sesión seguro, persistencia de sesión y envío de enlaces de restablecimiento de clave.
Base de Datos	Cloud Firestore (NoSQL)	Almacenamiento en la nube con actualización instantánea sin necesidad de recargar la página.
Backend API	Python 3, FastAPI, Uvicorn, Pydantic	Procesamiento matemático de valoración de inventario, márgenes de utilidad y cierres diarios.

4. Modelo de Base de Datos (Colecciones en Cloud Firestore)

La base de datos documental en **Cloud Firestore** (`tienda-tecnologia-inventario`) se organiza en 5 colecciones principales:

📁 PRODUCTOS (productos)

codigoBarras / **nombre** / **marca**	string
proveedorId / **proveedorNombre**	string (FK)
precioCompra / **precioVenta**	number (COP)
stockActual / **stockMinimo**	number
estadoStock	OK | BAJO | AGOTADO
serializado / **serialesDisponibles**	bool / array

📁 MOVIMIENTOS / KARDEX (movimientos)

tipo	ENTRADA | SALIDA | AJUSTE
productoId / **productoNombre**	string (FK)
cantidad	number
stockAnterior → stockNuevo	number
proveedorNombre / **cliente**	string
seriales / **fechaOperacion**	array / string

📁 VENTAS (ventas)

numero / **fechaVenta**	number / string
cliente / **vendedorNombre**	string
items (detalle + seriales vendidos)	array
total / **estado**	number / COMPLETADA

📁 USUARIOS (usuarios) & PROVEEDORES (proveedores)

usuarios.uid / **nombre** / **correo**	string (PK)
usuarios.rol / **activo**	admin | operador / bool
proveedores.nombre / **nit**	string
proveedores.telefono / **correo**	string

5. Descripción de Módulos y Vistas del Sistema

🔐 5.1. Módulo de Acceso y Seguridad (Login.jsx y AuthContext.jsx)

- **Inicio de sesión y Registro:** Permite autenticar usuarios existentes o registrar nuevos colaboradores (asignando por seguridad el rol de `operador` por defecto).
- **Recuperación de contraseña:** Envía un enlace oficial al correo electrónico del usuario para restablecer su clave.
- **Verificación de cuenta activa:** Si un administrador desactiva a un empleado, el sistema bloquea su acceso de inmediato.

📊 5.2. Panel General / Dashboard (Dashboard.jsx)

- Muestra indicadores clave en tiempo real: **Productos Registrados**, **Unidades en Vitrina**, **Valoración Total a Costo** y **Alertas de Stock**.
- Incluye un **Banner de Atención Inmediata** que destaca los artículos agotados o por debajo del stock mínimo configurado.

📦 5.3. Catálogo de Productos (Products.jsx)

- Creación de productos permitiendo formato numérico con puntos de miles (ej. `650.000`, `1.200.000`).
- Buscador en vivo por nombre o código de barras y filtro rápido por estado (`OK`, `BAJO`, `AGOTADO`).
- **Edición y Eliminación protegida:** Solo los usuarios con rol `admin` visualizan y pueden ejecutar los botones de **Editar** y **Eliminar**.

🔄 5.4. Historial de Movimientos, Entradas y Ventas (Movements.jsx)

- **Consultar Historial (Kardex):** Tabla auditable con filtros por fecha, producto, serial y tipo de operación (`ENTRADA`, `SALIDA`, `AJUSTE`), mostrando la transición exacta (*Stock Anterior* ➔ *Stock Nuevo*).
- **Registrar Entrada de Mercancía:** Recepción de pedidos seleccionando producto, proveedor, fecha de ingreso, cantidad y escaneo de seriales/IMEIs, sumando automáticamente las unidades al stock actual.
- **Registrar Venta con Escáner:** Punto de venta que permite agregar productos mediante lector de código de barras o selector, asignar los seriales entregados al cliente y descontar automáticamente las existencias.

🚚 5.5. Proveedores (Suppliers.jsx), 📈 5.6. Reportes (Reports.jsx) y 👥 5.7. Usuarios (UsersManagement.jsx)

- **Directorio de Proveedores:** Gestión de distribuidores autorizados con Razón Social, NIT, teléfono y correo electrónico.
- **Reportes Financieros y Cierre Diario:** Conectado al microservicio en Python FastAPI (`http://localhost:8000`) para calcular costo total, venta potencial, ganancia proyectada y cierre diario.
- **Gestión de Colaboradores:** Exclusivo para el Administrador; permite activar o suspender cuentas de empleados y alternar roles entre `Operador` y `Administrador`.

6. Matriz de Roles y Permisos

Funcionalidad / Vista del Sistema	Administrador (`admin`)	Operador / Empleado (`operador`)
Iniciar sesión y recuperar contraseña por correo	✅ Permitido	✅ Permitido
Ver Panel General (Dashboard) y Alertas de Stock	✅ Permitido	✅ Permitido
Consultar, filtrar y crear productos en el Catálogo	✅ Permitido	✅ Permitido
Registrar Entrada de Mercancía con Proveedor y Seriales	✅ Permitido	✅ Permitido
Registrar Ventas con Escáner de Código de Barras y Seriales	✅ Permitido	✅ Permitido
Consultar Historial de Movimientos (Kardex Auditable)	✅ Permitido	✅ Permitido
Editar datos, stock o precios de un producto existente	✅ Permitido	❌ Oculto y bloqueado
Eliminar / dar de baja productos del catálogo	✅ Permitido	❌ Oculto y bloqueado
Gestionar usuarios (Activar/Desactivar cuentas y cambiar roles)	✅ Permitido	❌ Oculto y bloqueado

7. Estructura de Carpetas del Proyecto

```
TecnoStock_web/
├── iniciar_proyecto.bat # Lanzador automático de Backend y Frontend en Windows
├── README.md # Documentación principal del repositorio
├── backend/
│ ├── main.py # API REST en Python FastAPI (Valoración y Cierre Diario)
│ └── requirements.txt # Dependencias de Python (FastAPI, Uvicorn, Pydantic)
├── frontend/
│ ├── index.html # Plantilla raíz en español
│ └── src/
│ ├── App.jsx # Enrutador principal de vistas y protección de sesión
│ ├── context/ → AuthContext.jsx # Estado global de usuario y rol
│ ├── firebase/ → config.js # Conexión a Firebase Auth y Firestore
│ ├── components/ → Sidebar.jsx, Navbar.jsx, ErrorBoundary.jsx
│ └── pages/ → Login.jsx, Dashboard.jsx, Products.jsx, Movements.jsx, Suppliers.jsx,
Reports.jsx, UsersManagement.jsx
└── docs/ # Cronograma de Sprints, Evidencias y Contexto del Proyecto
```